

SIR SYED UNIVERSITY OF ENGINEERING & TECHNOLOGY
COMPUTER SCIENCE & INFORMATION TECHNOLOGY DEPARTMENT

Fall 2024
Compiler Construction (CS-310)
Assignment # 01

Semester:5th
Announced Date:15/11/2024
Total Marks:03
Instructor Name: **Ms. Mehwish Wahid**

Batch: 2022
Due Date:20/11/2024
Marks Obtained:

Name: Muhammad Faisal
Roll Number: 2022F-BCS-152
Section: C

CLO #	Course Learning Outcomes (CLOs)	PLO Mapping	Bloom’s Taxonomy
CLO 01	Discuss the fundamental concepts of the basic structure of compiler and recall R.E and FA.	PLO-01 (Academic Education)	C1 (Understanding)

Q1) Explain the role of a compiler in program execution. How does it differ from an interpreter? Provide examples of situations where each would be preferred. Additionally, discuss the modern accessories of compilers and their benefits.

Role of a Compiler in Program Execution

A compiler is a program that translates high-level source code written by a programmer into machine code (binary instructions) that can be executed by a computer's hardware. This process involves several stages, including lexical analysis, syntax analysis, semantic analysis, optimization, and code generation.

The compiled machine code is stored as an executable file, which can be run independently without requiring the source code or the compiler.

Key Differences Between a Compiler and an Interpreter

Feature	Compiler	Interpreter
Translation	Converts the entire source code into machine code at once.	Translates and executes the source code line by line.
Output	Produces an executable file.	Does not produce an executable; runs code directly.
Execution Speed	Faster execution (compiled code is optimized and pre-translated).	Slower (each line is translated and executed during runtime).
Errors	Displays all errors after compilation.	Stops and reports errors as they occur.
Examples	C, C++, Rust	Python, JavaScript, PHP

Situations Where Each is Preferred

- **Compiler Preferred:**
 - Applications requiring high performance (e.g., system software, games).
 - Programs that need to run on multiple devices without source code (e.g., commercial software).
 - When optimization for speed and efficiency is critical.
- **Interpreter Preferred:**
 - Rapid prototyping or scripting (e.g., web development with JavaScript).
 - Educational purposes to debug code easily and interactively.
 - Cross-platform script execution without recompilation (e.g., Python for data analysis).

Modern Accessories of Compilers

Modern compilers often come with advanced tools and features to enhance developer productivity:

- 1. **Integrated Development Environments (IDEs):**
 - Compilers integrated with editors, debuggers, and profilers (e.g., Visual Studio, Eclipse).
 - Benefit: Streamlines coding, testing, and debugging.
- 2. **Just-In-Time (JIT) Compilation:**
 - Combines compilation and interpretation (e.g., Java Virtual Machine, .NET CLR).
 - Benefit: Optimizes runtime performance by compiling frequently used code on-the-fly.
- 3. **Cross-Compilers:**
 - Compilers that generate machine code for a different platform than the one they run on (e.g., ARM compilers for embedded systems).
 - Benefit: Essential for embedded and IoT device development.
- 4. **Static Analysis Tools:**
 - Integrated tools that analyze code for potential errors, security vulnerabilities, or inefficiencies (e.g., Clang Static Analyzer).
 - Benefit: Improves code quality and security.
- 5. **Code Optimization Levels:**
 - Provides options to optimize for speed, size, or debugging ease (e.g., GCC’s -O1, -O2, -O3 flags).
 - Benefit: Tailors the compiled program to specific requirements.
- 6. **Language Support and Extensions:**
 - Many compilers support multiple languages and their extensions (e.g., LLVM for C, C++, Swift).
 - Benefit: Flexibility for multi-language projects.

Q2) Perform lexical analysis on the following code. Classify the lexemes, patterns and tokens. Also mention the attribute values where required.

```
import java.util.Scanner;
public class TableExample
{
    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number: ");
        int num = sc.nextInt();           //reading a number whose table is to be print
        for (int i = 1; i <= 10; i++)    //loop start execution form and execute until the
                                        //condition i<=10 becomes false
        {
            System.out.println(num + " * " + i + " = " + num * i); //prints table of the
                                                                    entered number
        }
    }
}
```

Lexical Analysis of the Given Code

Lexical analysis involves breaking the input code into **lexemes** (smallest meaningful units), categorizing them as **tokens** with associated **patterns**, and assigning **attribute values** where required. Below is the analysis:

Lexeme	Token	Pattern	Attribute Value
import	KEYWORD	Matches the reserved word import.	None
java.util.Scanner	IDENTIFIER	Matches a valid identifier pattern.	Fully qualified class name
public	KEYWORD	Matches the reserved word public.	None
class	KEYWORD	Matches the reserved word class.	None
TableExample	IDENTIFIER	Matches a valid identifier pattern.	Name of the class: TableExample.
{	SYMBOL	Matches {.	None
public	KEYWORD	Matches the reserved word public.	None
static	KEYWORD	Matches the reserved word static.	None
void	KEYWORD	Matches the reserved word void.	None
main	IDENTIFIER	Matches a valid identifier pattern.	Name of the method: main.
(	SYMBOL	Matches (.	None
String	KEYWORD	Matches the reserved word String.	None
args	IDENTIFIER	Matches a valid identifier pattern.	Parameter name: args.
[]	SYMBOL	Matches [].	Denotes an array.
)	SYMBOL	Matches).	None
{	SYMBOL	Matches {.	None

Scanner	IDENTIFIER	Matches a valid identifier pattern.	Class name: Scanner.
sc	IDENTIFIER	Matches a valid identifier pattern.	Object name: sc.
=	OPERATOR	Matches assignment operator =.	None
new	KEYWORD	Matches the reserved word new.	None
Scanner	IDENTIFIER	Matches a valid identifier pattern.	Class name: Scanner.
(	SYMBOL	Matches (.	None
System.in	IDENTIFIER	Matches a valid identifier pattern.	Standard input stream.
)	SYMBOL	Matches).	None
;	SYMBOL	Matches ;.	End of statement.
System.out.print	IDENTIFIER	Matches a valid identifier pattern.	Method call for printing.
(	SYMBOL	Matches (.	None
"Enter number: "	STRING_LITERAL	Matches a quoted string.	Value: "Enter number: ".
)	SYMBOL	Matches).	None
;	SYMBOL	Matches ;.	End of statement.
int	KEYWORD	Matches the reserved word int.	None
num	IDENTIFIER	Matches a valid identifier pattern.	Variable name: num.
=	OPERATOR	Matches assignment operator =.	None
sc.nextInt()	IDENTIFIER	Matches a valid identifier pattern.	Method call for reading integer input.
;	SYMBOL	Matches ;.	End of statement.
for	KEYWORD	Matches the reserved word for.	None
(	SYMBOL	Matches (.	None
int	KEYWORD	Matches the reserved word int.	None
i	IDENTIFIER	Matches a valid identifier pattern.	Variable name: i.
=	OPERATOR	Matches assignment operator =.	None
1	NUMERIC_LITERAL	Matches a number.	Value: 1.
;	SYMBOL	Matches ;.	None
i	IDENTIFIER	Matches a valid identifier pattern.	Variable name: i.
<=	OPERATOR	Matches relational operator <=.	None
10	NUMERIC_LITERAL	Matches a number.	Value: 10.
;	SYMBOL	Matches ;.	None
i++	OPERATOR	Matches increment operator ++.	None
)	SYMBOL	Matches).	None
{	SYMBOL	Matches {.	None
System.out.println	IDENTIFIER	Matches a valid identifier pattern.	Method call for printing with a newline.
(	SYMBOL	Matches (.	None
num+" * "+i+" = "+num*i	EXPRESSION	Matches an expression.	Value: Concatenation and multiplication.
)	SYMBOL	Matches).	None
;	SYMBOL	Matches ;.	End of statement.
}	SYMBOL	Matches }.	None
}	SYMBOL	Matches }.	None
}	SYMBOL	Matches }.	None